Алгоритм Борувки на SPLA и Apache Spark

Шальнев Владислав, Кадочников Даниил

SPLA — разреженная линейная алгебра

SPLA (репозиторий, C++ API) — MIT, **разреженная линейная алгебра** на **GPU (OpenCL)** и **CPU**.

- **Производительность**: типовые ядра (**m×v**, редукции, маски) на GPU эффективнее ручных циклов по графу
- **Semiring**: одни и те же примитивы под разные задачи — свои `×` / `+` и типы
- **Портируемость**: OpenCL, выбор устройства; API **C/C++** и **Python (pyspla)**

Apache Spark

Apache Spark — фреймворк для распределённой обработки больших данных

Ключевые особенности:

- Вычисления **in-memory** (в оперативной памяти)
- Автоматический **параллелизм** на кластере машин

GraphX — модуль для работы с графами:

- Автоматическое распределение графа по узлам
- Итеративные алгоритмы в стиле **Pregel**
- API для графовых операций (`aggregateMessages`, `connectedComponents`)

Алгоритм Борувки для MSF

Задача: найти **минимальный остовный лес** (MSF) взвешенного графа.

Шаги одной итерации:

1. Для каждой компоненты найти минимальное исходящее ребро
2. Добавить найденные рёбра в MSF
3. Объединить компоненты, соединённые выбранными рёбрами

Повторяем, пока не закончатся межкомпонентные ребра.

Борувка в терминах линейной алгебры

Данные:

- S — матрица смежности
- кортеж ребра $(w,\ i)$: w — вес, i — **индекс соседа** (столбец)
- $parent[v]$ — корень компоненты вершины v
- $edge[v]$ — минимальное исходящее ребро из вершины v
- $cedge[r]$ — минимум среди $edge[\cdot]$ по вершинам компоненты с корнем r

Полукольцо для шага «минимум по строке»:

- $\otimes = \mathrm{FIRST}$ — в произведении берётся первый операнд (значение из S)
- $\oplus = \mathrm{MIN}$

Борувка в терминах линейной алгебры

Инициализация: $S = A, parent[v] = v$

while $S \neq \emptyset$:

1. $edge = \mathrm{mxv}(S,\ \mathbf{1},\ \otimes{=}\mathrm{FIRST},\ \oplus{=}\mathrm{MIN})$
2. $cedge[parent[i]] = \min\big(cedge[parent[i]],\ edge[i]\big)$
   — агрегация на корнях компонент
3. Обновить $parent$: корни переходят к соседней компоненте
4. Сжатие путей: обновление $parent$ для каждой вершины
5. $S = select(S,\ parent[i] \neq parent[j])$
   — удалить внутрикомпонентные рёбра

Борувка в терминах Apache Spark

Инициализация:

Graph[Long, Short]

- vertices: VertexRDD[Long] — (vertexId, componentId)
- edges: EdgeRDD[Short] — (src, dst, weight)

currentGraph = graph.**mapVertices**((vid, _) => vid)

Шаг 1: Минимальное ребро для каждой вершины

cheapestPerVertex = currentGraph.**aggregateMessages**(
sendMsg: если компоненты разные → отправить ребро,
mergeMsg: выбрать минимальное
)

Борувка в Apache Spark

Шаг 2: Минимальное ребро для компоненты

cheapestPerComponent = cheapestPerVertex
.**innerJoin**(vertices) — добавить componentId
.**reduceByKey**(min) — минимум по компоненте

Шаг 3: Дедупликация рёбер

finalEdges = cheapestPerComponent
.**map**(нормализовать пары компонент)
.**reduceByKey**(min) — одно ребро на пару (A,B)

Борувка в Apache Spark

Шаг 4: Объединить компоненты

Вариант A (гибридный):

updates = **Union-Find** на driver + parallelize обратно

Вариант B (Spark-only):

updates = Graph(components, edges).**connectedComponents**()

Борувка в Apache Spark

Шаг 5: Применить обновления

vertexToNewComp = vertices

.**map**(vertexId, oldComp) → (oldComp, vertexId)

.**join**(updates)

.**map** → (vertexId, newComp)

currentGraph = currentGraph

.**outerJoinVertices**(vertexToNewComp) — обновить componentId

.**subgraph**(оставить только межкомпонентные рёбра)

Эксперимент 1

Цель: сравнить время работы алгоритма Борувки на SPLA (No Acc, Intel UHD, POCL, NVIDIA) и Apache Spark на реальных графах разного размера

Эксперимент 2

Цель: исследовать влияние типа графа на относительную производительность реализаций алгоритма Борувки на SPLA (CPU) и Apache Spark

Вопрос: зависит ли соотношение времени работы SPLA и Spark от структурных свойств графа (дорожная сеть, триангуляция, социальная сеть)?

Ход экспериментов

Эксперимент 1 — влияние размера графа

- 15 запусков алгоритма на каждом графе и библиотеке, 5 прогревов
- Вычисление среднего времени и доверительных интервалов
- Визуализация и анализ результатов

Эксперимент 2 — влияние типа графа

- 5 запусков алгоритма на каждом графе и библиотеке, 3 прогрева
- Сравнение SPLA (CPU) и Spark
- Визуализация и анализ результатов

Данные Эксперимент 1

Граф	Количество вершин	Количество рёбер
amazon0601	403 394	2 443 408
citationCiteseer	268 495	1 156 647
coPapersDBLP	540 486	15 245 729
in-2004	1 382 908	13 591 473
cit-Patents	3 774 768	16 518 947

Источники: A High-Performance MST Implementation for GPUs 2023, SuiteSparse Matrix Collection

Данные Эксперимент 2

Данные эксперимента: Road Networks

Граф	Вершины	Рёбра
road-usroads	129 164	165 435
USA-road-d.NY	264 346	730 100
USA-road-d.COL	435 666	1 042 400
road-belgium-osm	1 038 823	1 549 970
road-roadNet-CA	1 957 027	2 760 388

Данные эксперимента: Geometric Graphs

Граф	Вершины	Рёбра
delaunay_n17	131 072	393 176
delaunay_n18	262 144	786 396
delaunay_n19	524 288	1 572 823
delaunay_n20	1 048 576	3 145 686
delaunay_n21	2 097 152	6 291 408

Данные эксперимента: Social & Collaboration Networks

Граф	Вершины	Рёбра
soc-sign-epinions	131 828	840 799
ca-citeseer	227 320	814 134
soc-flickr	513 969	3 190 452
soc-lastfm	1 191 805	4 519 330
soc-flickr-und	2 394 385	15 555 042

Источник: Network Repository (NRVIS)

Характеристики машины Эксперимент 1

ОС: Ubuntu 24.04.4 LTS

CPU: Intel Core i5-10300H @ 2.50 ГГц

- 4 ядра, 8 потоков (Hyper-Threading)
- L1: 256 КБ, L2: 1 МБ, L3: 8 МБ

GPU:

- NVIDIA GeForce GTX 1650 Ti Mobile (4 ГБ GDDR6);
- Intel UHD Graphics (Comet Lake-H GT2), встроенная

RAM: 16 ГБ DDR4, 3200 МГц

Характеристики машины Эксперимент 2

ОС: Windows 10 Pro (22H2)

CPU: 12th Gen Intel Core i5-1235U @ 1.30 GHz

- Ядра: 10 (2 производительных + 8 эффективных)
- Потоки: 12
- L3 Кэш: 12 МБ

RAM: 32 ГБ DDR4, 3200 МГц